Joshua Colcord

414-630-0515 | jcolcord@wisc.edu | jcolcord.me | github.com/joshcolcord | linkedin.com/in/joshcolcord

Education

University of Wisconsin–Madison Madison, WI

B.S. in Computer Sciences and Mathematics *Exp. May 2028*

- Transfer GPA: 3.475/4.00 (EECS & Mathematics, University of Minnesota)
- Relevant Coursework: Computer Architecture, Num. Theory, Combinatorial Optimization, Signals & Systems
- Organizations: International Collegiate Programming Contest (ICPC), Traders @ WI, UW Competitive Tennis

Experience

ADvanced Systems Laboratory (ADSL) Madison, WI

Undergraduate Researcher *Apr. 2026 – Present*

- Researched and built execution engine optimizations for GPU inference serving frameworks, focusing on memory efficiency and latency mitigation across multi-turn inference workloads. Advised by Dr. Remzi Arpaci-Dusseau.
- Applied Hawkes process modeling to capture burst-like arrival patterns in request traffic, implementing a C++/Go request scheduler to reduce TTFT, prefill, and queue-wait p99 latency by ∼6%.
- Designed GPU KV cache allocation and eviction policies using a predictive TTL algorithm, improving throughput by 20% under cache saturation during multi-turn workloads.
- Implemented prefix-aware chunk preservation for multi-turn requests, reducing KV page fragmentation and swap I/O overhead by ∼8%.

Voyager AI Milwaukee, WI

Backend Software Engineering Intern *May 2026 – Aug. 2026*

- Architected and optimized distributed backend infrastructure for a production data ingestion service using Python, Docker, and AWS Fargate, implementing dynamic scaling, caching layers, and worker concurrency.
- Designed multi-thread, multi-worker per-object chunking routines for data ingestion hotpaths, reducing compute cost by 40% and p50 time-to-completion by ∼15%.
- Profiled and eliminated state-based race conditions across AWS Fargate worker instances, reducing production request latency by ∼300ms.
- Introduced a Redis caching layer for 25-type document classification lookups, eliminating redundant upstream inference calls and cutting average response time by ∼450ms.
- Engineered REST API endpoints and automated unit/integration test suites for backend microservices, shipping changes through CI/CD pipelines.
- Worked in an Agile team: participated in sprint planning, stand-ups, and code reviews, giving and incorporating feedback.

Skills

Languages: C++17/20, C, Python, Go, SQL, Java, Shell
Systems & Concurrency: Concurrency (Multithreading, Atomics), Lock-Free Data Structures, Latency Profiling, Memory Management, TCP/UDP, CMake
Cloud & Infrastructure: Linux, Docker, Kubernetes, AWS (Fargate, S3, EC2), Redis, PostgreSQL, REST APIs, CI/CD
AI/ML & Frameworks: vLLM, GPU KV-Cache Serving, NumPy, pandas, scikit-learn, XGBoost

Projects

Central Limit Order Book | *C++17/20, ITCH 5.0, Atomics, Multithreading* Feb. 2026 – Jul. 2026

- Built a high-throughput, price-time priority matching engine in C++ featuring a zero-copy binary parser for raw NASDAQ ITCH 5.0 market data feeds.
- Designed lock-free SPSC ring buffer for concurrent producer and consumer threads, using pointer operations to minimize memory copies under heavy order flow.
- Built benchmark harness w/ historical NASDAQ binary data to profile execution for order add and cancel paths.

XGBoost Tennis Model w/ K8s Homelab Ingestion | *Python, Kubernetes, PostgreSQL, XGBoost* Jul. 2026 – Present

- Built end-to-end ETL pipeline streaming and parsing live Polymarket order book state and tennis match feeds onto a self-hosted Kubernetes cluster, persisting to PostgreSQL with async workers for feature extraction.
- Built a Markov chain outcome prediction model with a backtesting engine and partial-Kelly sizing: designed an Avellaneda-Stoikov market-making strategy and identified lookahead and model leakage in backtests.

Slab Memory Allocator | *C++17, POSIX Threads, Atomics* Sep. 2026 – Present

- Building a concurrent memory allocator to replace standard malloc/free, using slab allocation to reduce fragmentation and contention under multi-thread load.